

第7讲 MATLAB数值积分与微分 (Numerical Integration and Differentiation)

7.1 数值积分 (Numerical Integration)

7.2 数值微分 (Numerical Differentiation)

7.1 数值积分 (Numerical Integration)

7.1.1 数值积分基本原理 (Basic Principles of Numerical Integration)

求解定积分的数值方法多种多样，如简单的梯形法(插值多项式是一次的)、辛普生(Simpson)法(插值多项式是二次的)等都是经常采用的方法。它们的基本思想都是将整个积分区间 $[a,b]$ 分成 n 个子区间 $[x_i, x_{i+1}]$ ， $i=1,2,\dots,n$ ，其中 $x_1=a$ ， $x_{n+1}=b$ 。这样求定积分问题就分解为求和问题。

7.1.2 数值积分的实现 (Realization of Numerical Integration)

1. 自适应积分法 (Adaptive integration method)

基于全局自适应积分算法，MATLAB提供了 **integral** 函数来求定积分。该函数的调用格式为：

I=integral('filename',a,b)

或 **I=integral(@filename,a,b)**

其中filename是被积函数名。a和b分别是定积分的下限和上限，积分限可以为无穷大。

Example 7-1 求定积分 $\int_0^{\infty} e^{-x^2} (\ln x)^2 dx$

fun=@(x) exp(-x.^2).*log(x).^2 %用匿名函数定义被积函数

q=integral(fun,0,inf) %这里的点乘不能少

q=

1.9475

2. 变步长辛普生法 (Variable Step Size Simpson Method)

基于变步长辛普生法，MATLAB给出了`quad`和`quadl`函数来求定积分。该函数的调用格式为：

`[I,n]=quad('filename',a,b,tol,trace)`

`[I,n]=quadl('filename',a,b,tol,trace)`

其中`filename`是被积函数名。`a`和`b`分别是定积分的下限和上限。`tol`用来控制积分精度，缺省时取`tol=10-6`。`trace`控制是否展现积分过程，若取非0则展现积分过程，取0则不展现，缺省时取`trace=0`。返回参数`I`即定积分值，`n`为被积函数的调用次数。

注意：`quadl`的精度更高，函数调用的步数要小于`quad`函数。

Example 7-2 求定积分 $\int_0^{3\pi} e^{-5x} \sin(x + \frac{\pi}{6}) dx$

(1) 建立被积函数文件fesin.m:

```
function f=fesin(x)
```

```
f=exp(-0.5*x).*sin(x+pi/6);
```

(2) 调用数值积分函数quad求定积分:

```
[S,n]=quad('fesin',0,3*pi)
```

S =

n =

0.9008

77

注意: 也可不建立被积函数文件, 使用语句函数
(内联函数) 求解。

```
g=inline('exp(-0.5*x).*sin(x+pi/6)');
```

```
[S,n]=quad(g,0,3*pi);
```

```
[S,n]=quadl(g,0,3*pi); %可以观察迭代次数
```

3.高斯-克朗罗德法（Gauss-Kronrod Method）

`[I,err]=quadgk('filename',a,b)`

`err`返回近似误差范围，积分上、下限可以是`-inf`或`inf`，也可以是复数。若上、下限是复数，则`quadgk`在复平面上求积分。

Example 7-3 求数值积分 $\int_0^{\pi} \frac{x \sin x}{1 + \cos^2 x} dx$

（1）被积函数文件`fsx.m`，命令如下：

```
function f=fsx(x)
```

```
f=x.*sin(x)./(1+cos(x).*cos(x));
```

（2）调用函数`quadgk`求定积分，命令如下：

```
I = quadgk(@fsx,0,pi)
```

```
I = 2.4674
```

4.梯形积分法（ Trapezoidal integration ）

在MATLAB中，对由表格形式定义的函数关系的求定积分问题用**trapz(X,Y)**函数。其中向量X,Y定义函数关系 $Y=f(X)$ 。

Example 7-4 用trapz函数计算定积分 $\int_1^{2.5} e^{-x} dx$

命令如下：

```
X=1:0.01:2.5;
```

```
Y=exp(-X);      %生成函数关系数据向量
```

```
trapz(X,Y)
```

```
ans =
```

```
0.2858
```

7.1.3 多重定积分的数值求解 (Numerical Solution of Multiple Definite Integrals)

使用MATLAB提供的`dblquad`函数就可以直接求出二重定积分的数值解。该函数的调用格式为：

`I=dblquad('f(x,y)',a,b,c,d,tol,trace)`

该函数求 $f(x,y)$ 在 $[a,b] \times [c,d]$ 区域上的二重定积分。

参数`tol`, `trace`的用法与函数`quad`完全相同。

三重积分的数值解，函数调用格式为：

`I=triplequad('f(x,y)',a,b,c,d,e,f,tol,trace)`

注意：这两个函数不返回被积函数的调用次数，如需要，可在被积函数中设置一个记数变量来统计。

Example 7-5 计算二重定积分 $\int_{-1}^1 \int_{-2}^2 e^{-x^2/2} \sin(x^2 + y) dx dy$

(1) 建立一个函数文件fxy.m:

```
function f=fxy(x,y)
```

```
global ki;
```

```
ki=ki+1;           %ki用于统计被积函数的调用次数
```

```
f=exp(-x.^2/2).*sin(x.^2+y);
```

(2) 调用dblquad函数求解:

```
global ki;
```

```
ki=0;
```

```
I=dblquad('fxy',-2,2,-1,1)
```

```
ki
```

```
I =
```

```
1.5745
```

```
ki =
```

```
1038
```

Example 7-6 计算三重定积分 $\int_0^1 \int_0^\pi \int_0^\pi 4xze^{-z^2y-x^2} dx dy dz$

```
fxyz=@(x,y,z) 4*x.*z.*exp(-z.*z.*y-x.*x);
```

```
triplequad(fxyz,0,pi,0,pi,0,1,1e-7)
```

7.2 数值微分 (Numerical Differentiation)

7.2.1 数值差分与差商 (Numerical Difference and Difference Quotient)

高等数学关心的是导函数的形式和性质，而数值分析关心的问题是怎样计算导函数 $f'(x)=g(x)$ 在一串离散点 $X=(x_1, x_2, \dots, x_n)$ 的近似值 $G=(g_1, g_2, \dots, g_n)$ 以及所计算的近似值有多大误差。

引进记号

$$\Delta f(x) = f(x+h) - f(x)$$

$$\nabla f(x) = f(x) - f(x-h)$$

$$\delta f(x) = f(x+h/2) - f(x-h/2)$$

称 $\Delta f(x)$ 、 $\nabla f(x)$ 和 $\delta f(x)$ 分别为函数在 x 点处以 $h(h>0)$ 为步长的向前差分、向后差分和中心差分。

称 $\Delta f(x)/h$ 、 $\nabla f(x)/h$ 和 $\delta f(x)/h$ 分别为函数在 x 点处以 $h(h>0)$ 为步长的向前差商、向后差商和中心差商。

当步长 $h(h>0)$ 充分小时，函数 f 在点 x 的微分接近于函数在该点的任意种差分。 f 在点 x 的导数接近于函数在该点的任意种差商。

7.2.2 数值微分的实现 (Implementation)

在MATLAB中，没有直接提供求数值导数的函数，只有计算向前差分的函数**diff**，其调用格式为：

$$\mathbf{DX}=\mathbf{diff}(\mathbf{X})$$

计算向量X的向前差分， $\mathbf{DX}(i)=\mathbf{X}(i+1)-\mathbf{X}(i)$ ， $i=1,2,\dots,n-1$ 。

$$\mathbf{DX}=\mathbf{diff}(\mathbf{X},n)$$

计算X的n阶向前差分。例如， $\mathbf{diff}(\mathbf{X},2)=\mathbf{diff}(\mathbf{diff}(\mathbf{X}))$ 。

$$\mathbf{DX}=\mathbf{diff}(\mathbf{A},n,\mathbf{dim})$$

计算矩阵A的n阶差分， $\mathbf{dim}=1$ 时(缺省状态)，按列计算差分； $\mathbf{dim}=2$ ，按行计算差分。

Example 7-7 生成以向量 $V=[1,2,3,4,5,6]$ 为基础的范得蒙矩阵，按列进行差分运算。

命令如下：

`V=vander(1:6)`

V=

1	1	1	1	1	1
32	16	8	4	2	1
243	81	27	9	3	1
1024	256	64	16	4	1
3125	625	125	25	5	1
7776	1296	216	36	6	1

DV=diff(V)

%计算V的一阶差分

DV =

31	15	7	3	1	0
211	65	19	5	1	0
781	175	37	7	1	0
2101	369	61	9	1	0
4651	671	91	11	1	0

Example 7-8 设 $f(x) = \sqrt{x^3 + 2x^2 - x + 12} + \sqrt[6]{x + 5} + 5x + 2$
用不同的方法求函数 $f(x)$ 的数值导数，并在同一个坐标系中做出 $f'(x)$ 的图像。

用三种方法：

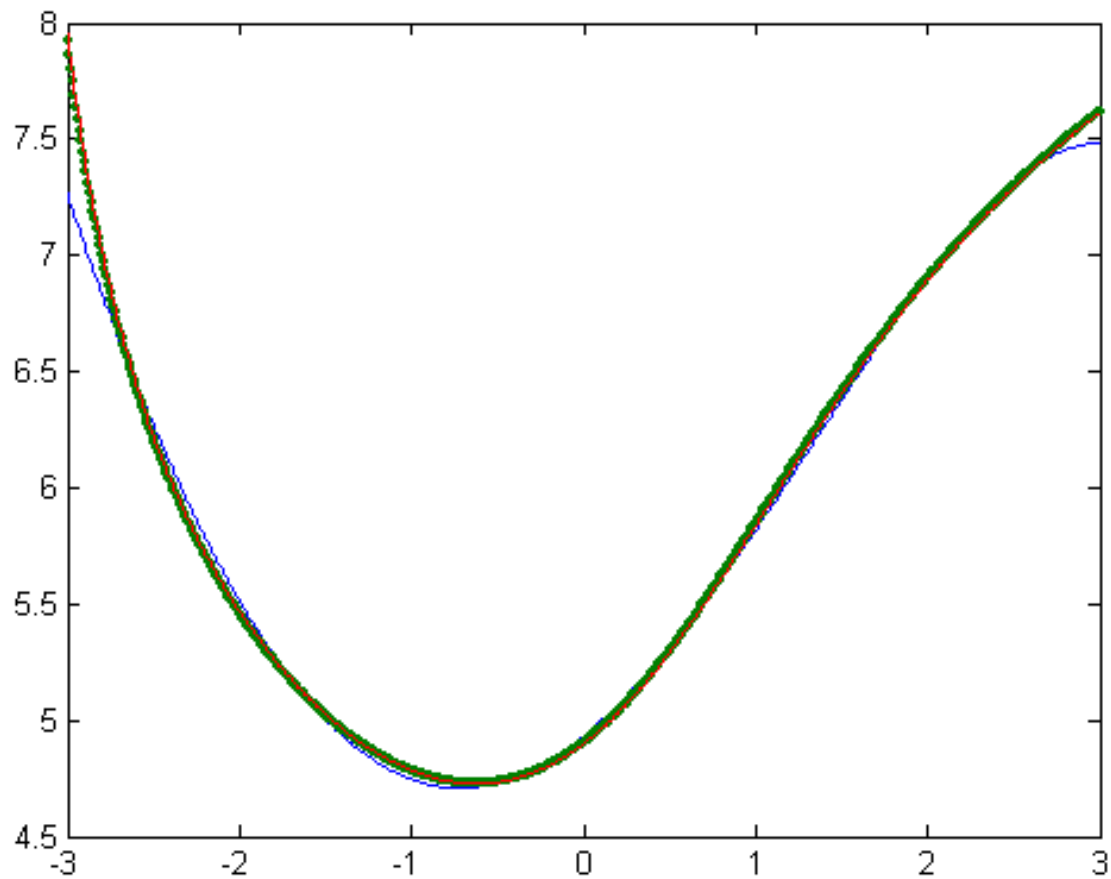
- 1、用一个5次多项式 $p(x)$ 拟合函数 $f(x)$ ，并对 $p(x)$ 求一般意义下的导数 $dp(x)$ ，求出 $dp(x)$ 在假设点的值；
- 2、直接求 $f(x)$ 在假设点的数值导数；
- 3、求出 $g(x)=f'(x)$ 的表达式

$$f'(x) = \frac{3x^2 + 4x - 1}{2\sqrt{x^3 + 2x^2 - x + 12}} + \frac{1}{6\sqrt[6]{(x + 5)^5}} + 5$$

然后求 $f'(x)$ 在假设点的数值导数。

程序如下：

```
f=inline('sqrt(x.^3+2*x.^2-x+12)+(x+5).^(1/6)+5*x+2');  
g=inline('(3*x.^2+4*x-1)./sqrt(x.^3+2*x.^2-  
    x+12)/2+1/6./(x+5).^(5/6)+5');  
x=-3:0.01:3;  
p=polyfit(x,f(x),5);           %1用5次多项式p拟合f(x)  
dp=polyder(p);                 %1对拟合多项式p求导数dp  
dpx=polyval(dp,x);             %1求dp在假设点的函数值  
dx=diff(f([x,3.01]))/0.01;     %2直接对f(x)求数值导数  
gx=g(x);                       %3求函数f的导函数g在假设点  
    的导数  
plot(x,dpx,x,dx,'.',x,gx,'-'); %作图
```



Ex: 求函数 $f(x) = \sqrt{x^2 + 1}$ 在指定点 $x = 1, 2, 3$ 的数值导数。